

# The two gates

SecureGate can stop a secret in two places: on your laptop, before a commit is saved, and on GitHub, before a pull request is merged. A **pull request** is a request to add a set of commits to the main version of the project; others can review it before it is merged.

## The laptop gate

A **pre-commit hook** is a check that Git runs just before it saves a commit. SecureGate's hook scans only the changes you are about to commit, which takes a second and works offline. Turn it on once, in the SecureGate folder:

```
make hooks
```

When it finds a secret to block, the commit stops and shows what to do:

```
Blocked:
demo_leak.py:1  acme****RoZ6
  why: provider-keys: A payment, cloud or private key gives direct access to money, data
or servers.
  fix: Treat it as leaked. Rotate (replace) the key at the provider now, remove it from
the code, and load it from an environment variable or a secrets manager instead.
```

`git commit --no-verify` skips the laptop gate. That is allowed on purpose: sometimes a commit has to go through, for example to show the merge gate at work. The merge gate still catches it.

## The merge gate

On GitHub, every pull request runs a **status check** called `secret-gate`. A status check is a test that GitHub shows next to a pull request, as a green tick or a red cross. This one is a **workflow**, a list of steps that GitHub Actions runs on its own computers: `.github/workflows/secret-gate.yml`.

It runs for every pull request to main, with four scanners: **Gitleaks** and **TruffleHog** read **every commit** in the pull request, not only the final result, because a secret that a later commit deletes is still in the history; **Semgrep** and **Bandit** read the code the pull request changed. Their findings are merged, and `policy.yaml` decides. Besides the check itself, the workflow posts one comment on the pull request with the same report (and updates that comment on every push), sends the results to the repository's **Security** tab, and keeps `findings.json` as a download of the run. The check turns red when a secret is blocked, and also when the scan could not run. Select **Details** next to the check, then **Summary** at the top left if you see a list of steps, to see the summary: a one-line verdict (BLOCKED, PASSED WITH WARNINGS or PASSED), a table with the policy rule, the file and line, the masked value, the scanners that found it and the live check, a checklist to rotate every blocked key, and why each warning was not blocked.

## Why only the merge gate is the real control

- The laptop gate only runs on computers where someone ran `make hooks`, and anyone can skip it with `--no-verify` or switch it off. It is a helpful early warning, not a control.

- The merge gate runs on GitHub, for every pull request, whoever opened it. With the setting below, nothing reaches main unless the check passed.
- It judges each pull request with the SecureGate, `policy.yaml`, `.gitleaks.toml`, `.trufflehog.yaml` and `rules/` of the base branch (usually main), so a pull request cannot loosen the rules that judge it. Changes to those files only count once they are merged. So a change to the gate itself ships in two pull requests: first the code, then the workflow that uses it.
- The job can read the code, write its own comment and upload results to the Security tab, and nothing else: it cannot push or merge.
- One thing it cannot prevent: a pull request that edits the workflow file itself, because GitHub runs the pull request's own copy of that file. So also require an approval in the ruleset (**Require approvals**), and read any pull request that touches `.github/` with care. If you work alone this is not possible, because nobody can approve their own pull request.

## The one-time setting, done by hand on GitHub

A **ruleset** is a set of rules GitHub enforces on branches. This one makes the check required:

1. Open the repository on GitHub, then **Settings**.
2. Choose **Rules**, then **Rulesets**, then **New ruleset**, then **New branch ruleset**.
3. Give it a name, such as "Protect main", and set **Enforcement status** to **Active**.
4. Under **Target branches**, choose **Add target** and add **main** (the default branch).
5. Tick **Require a pull request before merging**.
6. Tick **Require status checks to pass**, choose **Add checks** and pick **secret-gate**.
7. Keep the **Bypass list** empty, so that nobody, not even an admin, can skip the check.
8. Select **Create**.

**The check only appears in the list in step 6 after it has run at least once.** If `secret-gate` is not there, open a pull request first (the demo (demo-merge-gate.pdf) is a good one), wait until its check has finished, and try again.

## When the check is red

For a real key:

1. **Revoke it at the provider and create a new one.** This is the step that matters: the key is in the pull request's history, and anyone who can see the pull request can read it.
2. Store the new key in a secret manager and take the old one out of the code.
3. The check stays red as long as the key is in any commit of the pull request. The simplest way out is a new branch, made from main, with the change but without the key, and a new pull request for it.

For something that is not a secret, such as a test value: change `policy.yaml` so that it is ignored or only warned about, in a separate pull request. The change counts once that pull request is merged.